

LAB 1: Display Density Metrics & Image Array Mechanics in NumPy

Human-Computer Interaction (HCI) & Computer Graphics (CG)

Ebad Ur Rehman Khan • Roll No: 2K24/CSE/46

Task 1 — Display Pixel Density (PPI/DPI) Calculator

This script prompts the user for horizontal/vertical resolution and physical diagonal size, then computes total pixel count, simplified aspect ratio (via GCD), DPI, and a density classification.

Source Code

```
"""
Task 1: Display Pixel Density (PPI/DPI) Calculator
-----
Prompts the user for a screen's horizontal/vertical resolution and its
physical diagonal size, then computes:
- Total pixel count
- Simplified aspect ratio (using GCD)
- DPI / PPI
- Density classification
"""
import math

def calculate_display_metrics(w_px: int, h_px: int, d_inches: float) -> dict:
    """Compute total pixels, aspect ratio, DPI, and density category."""
    total_pixels = w_px * h_px

    # Simplify aspect ratio using the greatest common divisor
    divisor = math.gcd(w_px, h_px)
    aspect_ratio = f"{w_px // divisor}:{h_px // divisor}"

    # Diagonal pixel count (Pythagorean theorem) and DPI/PPI
    diagonal_px = math.sqrt(w_px ** 2 + h_px ** 2)
    dpi = diagonal_px / d_inches

    # Density classification
    if dpi < 100:
        category = "Low Density (Standard Monitor)"
    elif dpi <= 200:
        category = "Medium Density (HD Display)"
    else:
        category = "High Density (Retina / Mobile)"

    return {
        "total_pixels": total_pixels,
        "aspect_ratio": aspect_ratio,
        "dpi": round(dpi, 2),
        "category": category,
    }

def main():
    print("--- DISPLAY METRICS CALCULATOR ---")
    w_px = int(input("Enter horizontal resolution (pixels): "))
```

```

h_px = int(input("Enter vertical resolution (pixels): "))
d_inches = float(input("Enter physical diagonal size (inches): "))

metrics = calculate_display_metrics(w_px, h_px, d_inches)

print("\n--- DISPLAY METRICS ANALYSIS ---")
print(f"Total Pixel Count : {metrics['total_pixels']:,} pixels")
print(f"Aspect Ratio      : {metrics['aspect_ratio']}")
print(f"Calculated DPI       : {metrics['dpi']} DPI")
print(f"Density Category    : {metrics['category']}")

if __name__ == "__main__":
    main()

```

Output — MacBook Pro 16" (3456×2234, 16.2")

```

--- DISPLAY METRICS CALCULATOR ---
Enter horizontal resolution (pixels): 3456
Enter vertical resolution (pixels): 2234
Enter physical diagonal size (inches): 16.2

--- DISPLAY METRICS ANALYSIS ---
Total Pixel Count : 7,720,704 pixels
Aspect Ratio      : 1728:1117
Calculated DPI     : 254.02 DPI
Density Category   : High Density (Retina / Mobile)

```

Output — Google Pixel 6a (2400×1080, 6.1")

```

--- DISPLAY METRICS CALCULATOR ---
Enter horizontal resolution (pixels): 2400
Enter vertical resolution (pixels): 1080
Enter physical diagonal size (inches): 6.1

--- DISPLAY METRICS ANALYSIS ---
Total Pixel Count : 2,592,000 pixels
Aspect Ratio      : 20:9
Calculated DPI     : 431.44 DPI
Density Category   : High Density (Retina / Mobile)

```

Task 2 — Synthetic Image Matrix Creation

Builds a 300×400×3 uint8 array of zeros and fills each quadrant with a solid color using NumPy slicing, then reports the array's shape, dtype, size, and memory footprint.

Source Code

```
"""
Task 2: Environment Setup & Synthetic Image Matrix Creation
-----
Builds a 300x400x3 uint8 image array from scratch and fills each
quadrant with a solid color using array slicing.
"""
import numpy as np

def build_quadrant_image(height: int = 300, width: int = 400) -> np.ndarray:
    """Create an (H, W, 3) uint8 array with 4 solid-color quadrants."""
    img = np.zeros((height, width, 3), dtype=np.uint8)
    mid_h, mid_w = height // 2, width // 2

    img[0:mid_h, 0:mid_w] = [255, 140, 0] # Top-Left: Orange
    img[0:mid_h, mid_w:width] = [128, 0, 128] # Top-Right: Purple
    img[mid_h:height, 0:mid_w] = [0, 128, 128] # Bottom-Left: Teal
    img[mid_h:height, mid_w:width] = [255, 215, 0] # Bottom-Right: Gold/Yellow

    return img

def main():
    img = build_quadrant_image()

    print("--- SYNTHETIC MATRIX METRICS ---")
    print(f"Array Shape (H, W, C) : {img.shape}")
    print(f>Data Type : {img.dtype}")
    print(f"Total Elements : {img.size:,} values")
    print(f"Memory Footprint : {img.nbytes:,} bytes ({img.nbytes / 1024:.2f} KB)")

    from PIL import Image
    Image.fromarray(img).save("synthetic_quadrants.png")
    print("\nSaved visual output to synthetic_quadrants.png")

if __name__ == "__main__":
    main()
```

Terminal Output

```
--- SYNTHETIC MATRIX METRICS ---
Array Shape (H, W, C) : (300, 400, 3)
Data Type : uint8
Total Elements : 360,000 values
Memory Footprint : 360,000 bytes (351.56 KB)

Saved visual output to synthetic_quadrants.png
```

Generated Image



Figure: 300×400 quadrant image (Orange / Purple / Teal / Gold)

Task 3 — Channel Slicing & Isolation

Loads a synthetically generated sample image, extracts the R/G/B 2D intensity grids via Axis-2 slicing, builds single-channel color views, and displays a 2x3 matplotlib grid of colored and grayscale channel views.

Source Code

```
"""
Task 3: Channel Slicing & Isolation
-----
Loads sample.jpg, extracts the R/G/B channels via Axis-2 slicing, builds
single-channel color views, and displays a 2x3 matplotlib grid:
  Top row    -> Red-only, Green-only, Blue-only color images
  Bottom row -> Grayscale intensity maps for each channel
"""
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image

def load_image_array(path: str) -> np.ndarray:
    """Load an image file into an (H, W, 3) uint8 NumPy array."""
    return np.array(Image.open(path).convert("RGB"))

def extract_channels(img: np.ndarray):
    """Return the 2D R, G, B intensity grids using Axis-2 slicing."""
    red_channel = img[:, :, 0]
    green_channel = img[:, :, 1]
    blue_channel = img[:, :, 2]
    return red_channel, green_channel, blue_channel

def build_isolated_color(img: np.ndarray, channel_index: int) -> np.ndarray:
    """Return a 3-channel image with only `channel_index` active."""
    isolated = np.zeros_like(img)
    isolated[:, :, channel_index] = img[:, :, channel_index]
    return isolated

def main():
    img = load_image_array("sample.jpg")
    red, green, blue = extract_channels(img)

    red_only = build_isolated_color(img, 0)
    green_only = build_isolated_color(img, 1)
    blue_only = build_isolated_color(img, 2)

    print("--- CHANNEL EXTRACTION SUMMARY ---")
    print(f"Original Image Shape : {img.shape}")
    print(f"Red Channel 2D Shape : {red.shape} | Mean Intensity: {red.mean():.2f}")
    print(f"Green Channel 2D Shape: {green.shape} | Mean Intensity: {green.mean():.2f}")
    print(f"Blue Channel 2D Shape : {blue.shape} | Mean Intensity: {blue.mean():.2f}")

    fig, axes = plt.subplots(2, 3, figsize=(12, 7))

    axes[0, 0].imshow(red_only); axes[0, 0].set_title("Red-Only"); axes[0, 0].axis("off")
    axes[0, 1].imshow(green_only); axes[0, 1].set_title("Green-Only"); axes[0, 1].axis("off")
    axes[0, 2].imshow(blue_only); axes[0, 2].set_title("Blue-Only"); axes[0, 2].axis("off")

    axes[1, 0].imshow(red, cmap="gray"); axes[1, 0].set_title("Red Intensity"); axes[1, 0].axis(
"off")
    axes[1, 1].imshow(green, cmap="gray"); axes[1, 1].set_title("Green Intensity"); axes[1, 1].a
xis("off")
    axes[1, 2].imshow(blue, cmap="gray"); axes[1, 2].set_title("Blue Intensity"); axes[1, 2].axi
s("off")
```

```
plt.tight_layout()
plt.savefig("task3_channel_grid.png", dpi=150)
print("\nSaved figure to task3_channel_grid.png")
print("Display Window : Matplotlib 2x3 Subplot Grid Rendered.")

if __name__ == "__main__":
    main()
```

Terminal Output

```
--- CHANNEL EXTRACTION SUMMARY ---
Original Image Shape : (500, 700, 3)
Red Channel 2D Shape : (500, 700) | Mean Intensity: 117.25
Green Channel 2D Shape: (500, 700) | Mean Intensity: 165.96
Blue Channel 2D Shape : (500, 700) | Mean Intensity: 136.75

Saved figure to task3_channel_grid.png
Display Window : Matplotlib 2x3 Subplot Grid Rendered.
```

Matplotlib Figure

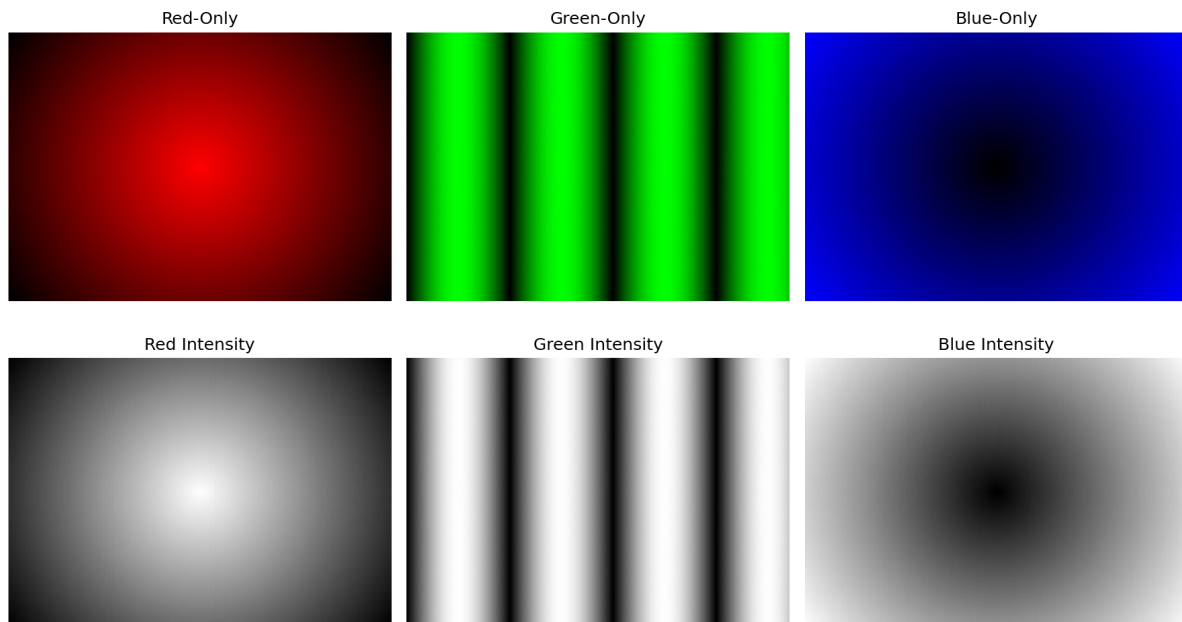


Figure: Top row = isolated color channels, bottom row = grayscale intensity maps

Task 4 — Spatial Downsampling & Pixelation via Striding

Downsamples the image by keeping every 10th pixel along rows/columns (NumPy striding), re-expands it with `np.repeat()` to visualize the pixelation, and reports the resulting dimension and memory savings.

Source Code

```
"""
Task 4: Spatial Downsampling & Pixelation via Striding
-----
Downsamples an image by taking every N-th pixel (striding), then
re-expands it back to the original size with np.repeat() to visualize
the resulting pixelation, and reports the size/memory savings.
"""
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image

N = 10 # step factor

def load_image_array(path: str) -> np.ndarray:
    return np.array(Image.open(path).convert("RGB"))

def downsample(img: np.ndarray, step: int) -> np.ndarray:
    """Keep every `step`-th pixel along rows and columns."""
    return img[::step, ::step, :]

def re_expand(small_img: np.ndarray, step: int) -> np.ndarray:
    """Blow the downsampled array back up using np.repeat on axes 0 and 1."""
    expanded = np.repeat(small_img, step, axis=0)
    expanded = np.repeat(expanded, step, axis=1)
    return expanded

def main():
    img = load_image_array("sample.jpg")
    small = downsample(img, N)
    expanded = re_expand(small, N)

    dim_reduction = (1 - 1 / N) * 100
    mem_savings = (1 - small.nbytes / img.nbytes) * 100

    print(f"--- DOWNSAMPLING ANALYSIS (N = {N}) ---")
    print(f"Original Shape      : {img.shape} | Memory: {img.nbytes:,} bytes")
    print(f"Downsampled Shape   : {small.shape} | Memory: {small.nbytes:,} bytes")
    print(f"Re-expanded Shape   : {expanded.shape} | Visual: Blocky Pixelation")
    print(f"Dimension Reduction : {dim_reduction:.2f}% reduction per axis")
    print(f"Memory Savings      : {mem_savings:.2f}% data reduction")

    fig, axes = plt.subplots(1, 2, figsize=(10, 5))
    axes[0].imshow(img); axes[0].set_title("Original"); axes[0].axis("off")
    axes[1].imshow(expanded[: img.shape[0], : img.shape[1]])
    axes[1].set_title(f"Re-expanded (N={N}) - Pixelated")
    axes[1].axis("off")
    plt.tight_layout()
    plt.savefig("task4_downsampling.png", dpi=150)
    print("\nSaved figure to task4_downsampling.png")

if __name__ == "__main__":
    main()
```

Terminal Output

```
--- DOWNSAMPLING ANALYSIS (N = 10) ---
Original Shape      : (500, 700, 3) | Memory: 1,050,000 bytes
Downsampled Shape   : (50, 70, 3) | Memory: 10,500 bytes
Re-expanded Shape    : (500, 700, 3) | Visual: Blocky Pixelation
Dimension Reduction  : 90.00% reduction per axis
Memory Savings       : 99.00% data reduction

Saved figure to task4_downsampling.png
```

Matplotlib Figure

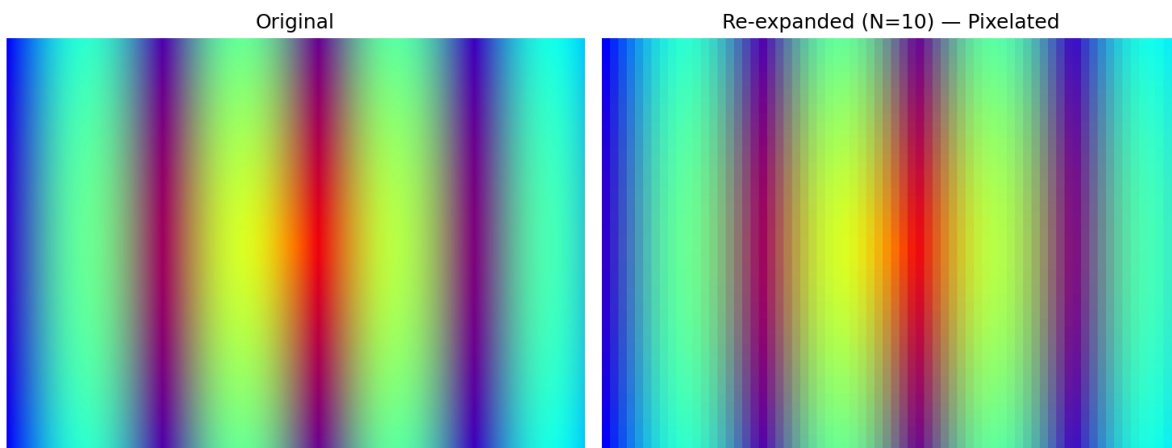


Figure: Original vs. re-expanded (N=10) pixelated image

Short Answer

Why does adding an Alpha channel (RGBA) increase an image array's memory consumption by 33% compared to standard RGB?

An RGB image stores 3 channels per pixel (Red, Green, Blue), while RGBA adds a 4th channel (Alpha, for transparency). Since each channel is typically stored as one byte (uint8) per pixel, memory usage scales directly with channel count: RGB uses 3 bytes/pixel and RGBA uses 4 bytes/pixel. Going from 3 to 4 channels adds 1 extra channel out of the original 3, i.e. $1/3 \approx 33\%$ more memory, even though the image resolution ($H \times W$) itself hasn't changed at all.